2026 Notebook

Daniel Jang

Table of contents

September **2**
New Methods 2
Mistakes I made 2

October **3**
Debiased Machine Learning (BIP Project) 3

The purpose of this yearly notebook is to document large scale progress in projects, any significant mistakes, important derivations, nice tricks that I think will serve me into the future, etc.

September

New Methods

M-estimation, Z-estimation, sandwich variance

Influence Functions, Efficient Influnece Functions, connections between IF/EIF and estimating functions

Mistakes I made

October

Debiased Machine Learning (BIP Project)

Debiased / Double Machine Learning (DML) is a semiparametric estimation method that allows us to obtain a $\sqrt{n}$-consistent and efficient estimate for a low-dimensional target parameter ψ (perhaps to do statistical inference) while accounting for many covariates Z with an unknown functional form. DML originated from the economics literature, but biostatistics has been slowly adopting it. We need to look no further than one of the papers that Jinbo made us present, which used DML as one of the estimation methods for RCTs. My goal is to give a high level explanation of the method, giving important results without going deep into their proofs.

I will restrict our discussion on DML to partially linear regression models, since it will connect nicely to concepts we learned this semester. Let A be a binary treatment whose effect is of primary scientific interest, let Y be our continuous outcome (we can extend this beyond continuous outcomes), and let $Z = (Z_1, ... Z_p)$ be a p-dimensional vector of covariates that are not of primary scientific interest. Finally, let $f : \mathcal{Z} \to \mathbb{R}$, where $\mathcal{Z}$ is the support of Z and the set of functions lives in the L_2 Hilbert Space. What this means is that this guarantees any function of Z has a finite second moment, which implies that $\text{Var}(f(Z)) < \infty$. Lastly, we say that L_2 is an infinite dimensional space. As an informal proof, consider the subset of all polynomials of L_2 and convince yourself that there is no finite basis that spans the space of polynomial functions.

The partially linear regression model (with one target parameter, and where the intercept term β_0 is absorbed into the nonparametric part) is given as:

$$Y = \beta_A A + f(Z) + \varepsilon$$

For us to obtain our estimate of interest $\hat{\beta}_A$, we need to estimate $f(Z)$. The naive approach is to let some machine learning estimator to estimate $\beta_A A + f(Z)$ jointly to give us $\hat{\beta}_A A + \hat{f}(Z)$. For example, we can use something like **kernel regression** or **spline regression**, which are nonparametric data-driven methods to find a plausible and generalizable functional form for the covariates. Another example, which comes nicely from the **LASSO** presentation we had, is in situations where Z is sparse, i.e. very few covariates are actually predictive of the outcome. We can specify $f(Z) = \eta_1 Z_1 + ... \eta_p Z_p$ and use LASSO, which penalizes each parameter estimate (including $\hat{\beta}_A$ in this case) towards zero to different extents. This isn't strictly a partially linear regression model, but is motivated from the issue that we have nuisance parameters that must be estimated, but something about them is preventing us from using more basic statistical methods. In this example, when $p > n$ or close to it.

What might go wrong with this approach? Regularization Bias. The methods I mentioned above are useful for prediction, but the core tenant of machine learning is the bias-variance tradeoff. Some amount of bias in the parameter estimates may be accepted if it helps reduce the variance of the model, and both aforementioned methods suffer from regularization bias. There is a lot of theory behind how regularization bias leads to estimators that are not $\sqrt{n}$-consistent, but I will skip those details. In the naive approach, there is no guarantee that our estimate $\hat{\beta}_A$ did not undergo some amount of regularization, which means it is likely biased. As we learned, this becomes problematic for interpreting the point estimate and for statistical inference.

How do we overcome regularization bias? Orthogonalization with the residuals. This idea effectively de-biases the biased estimate we get from the naive method. Let's use the fully parametric analog from linear regression as the starting point. Recall the **Frisch-Waugh-Lovell (FWL) Theorem**: For a linear regression model, where X_{-k} denotes all predictors other than X_k,

$$\mathbb{E}(Y \mid X) = \beta_0 + \boxed{\beta_k} X_K + X_{-k}^\top \beta_{-k} + \varepsilon \implies Y^* = \boxed{\beta_k} X_k^* + \varepsilon^*$$

where $Y^* = Y - \hat{\mathbb{E}}(Y \mid X_{-k})$ are the residuals of Y after removing all the effects from the vector of predictors X_{-k}, and $X_k^* = X_k - \hat{\mathbb{E}}(X_k \mid X_{-k})$ are the residuals of X after removing all the effects from the vector of predictors X_{-k}, and ε^* is proportional to the residual-maker matrix for the remaining predictors.

The same theorem applies to the partially linear regression model. We have two auxiliary prediction problems that require us to estimate two models (this is why there is the word "double" in "double machine learning") without a specified functional form:

$$\text{Outcome Model} \implies Y^* = Y - \hat{\mathbb{E}}[Y \mid Z]$$
$$\text{Treatment Model} \implies A^* = A - \hat{\mathbb{E}}[A \mid Z]$$

Thus, the FWL theorem gives us $Y^* = \hat{\beta}_A A^* + \varepsilon^*$, and by linear models homework 1 (regression-through-the-origin model), we can rewrite the DML estimator as:

$$\boxed{\hat{\beta}_A = \frac{\sum_{i=1}^n [Y_i^* A_i^*]}{\sum_{i=1}^n A_i^{*2}}}$$

where $\hat{\beta}_A$ is a $\sqrt{n}$-consistent, efficient estimator of β_A, under certain conditions on the convergence rate of the machine learning estimators (to anyone who's curious, it's $o(n^{-1/4})$). This is unfortunately a leap of faith you will have to take, since the details require some knowledge on functional analysis and methods for estimation from Inference II, such as Z-estimation and

Taylor expansions. The authors in the original paper called the reason why we get an estimator which such good properties **Neyman Orthogonality**.

How do we estimate the two models to obtain $\hat{\beta}_A$? It's in the method's name: *Machine Learning*. Neyman Orthogonality provides flexibility in which machine learning estimator that can be used. Unfortunately, we can't use any machine learning method for the estimation of $\hat{f}$ for every situation. Again, the details are a bit beyond the scope of the presentation, but the method must converge faster than the $o(n^{-1/4})$ rate.

Finally, there is one more piece to DML estimation for partially linear regression: **cross-fitting to avoid overfitting**. For example, estimating $\hat{\mathbb{E}}[Y \mid Z]$ depends on all the sample points of Y. Thus, computing residuals means that the data are too reliant on themselves. Instead, the authors' guidance is to use **K-fold cross-fitting** to avoid wasting data via a training-testing split (which dramatically reduces parameter efficiency). The idea (for those unfamiliar) is to divvy up your data into K groups (called folds) so that we create a clear division in which data are used for both steps. Iterating through all the folds (the j^{th} fold), the nuisances $\hat{\mathbb{E}}[Y \mid Z]$ and $\hat{\mathbb{E}}[Y \mid A]$ are estimated on all folds except for the j^{th}, and residuals are calculated for those in the j^{th} fold. What we are left with is residuals for our entire data. These residuals are not overfit because the nuisances were estimated with "different" data.

We finally have a procedure to get our $\sqrt{n}$-consistent and efficiency treatment effect from a semiparametric model. Now that we did all this hard work, we should try and implement it in a real dataset. I will motivate it using RCT data from (https://bingkai-wang.github.io/RCT_Bench-main/website/). As another caveat, incorporation of nonparametric methods often requires a higher sample size than its correctly-specific parametric counterpart.

The reason I chose an observational dataset instead of an RCT is because the randomization scheme helps account for both measured and unmeasured confounders. Thus, the difference between DML and the parametric estimation will be the efficiency gain by the addition of more precision variables. On the other hand, an observational dataset does not have randomization, and therefore, confounders and their relationship are unknown. I want to create a contrast between baseline covariate adjustment in a parametric linear model and being able to account for everything and letting the method adjust to provide an unbiased and efficient treatment effect estimate.

Start here: I am trying to determine which trial will work best for demonstration. MAYBE NOT TRIAL DATA BECAUSE RANDOMIZATION. CHOOSE AN OBSERVATIONAL DATASET IN WHICH SOMEONE WROTE A PAPER THAT FIT A PARAMETRIC MODEL

```
# Load in DML package and RCT data
library(DoubleML)
library(nhanesA)
```

Some data visualization and identification of treatment + covariates

Recreation of paper analysis for treatment effect + parametric assumptions

Debiased Machine Learning Estimator via a partially linear regression model

Comparison of Treatment Effects: did it make much of a difference at all?

Some references:

1) The canonical paper: Double/Debiased Machine Learning for Treatment and Structural Parameters1 Victor Chernozhukov†, Denis Chetverikov‡, Mert Demirer† Esther Duflo†, Christian Hansen§, Whitney Newey†, James Robins
2) How should covariates be handled in randomized trials? Empirical evidence from 50 trials and recommendations for practice Yulin Shao1, Liangbo Lyu1, Menggang Yu1, Bingkai Wang1,
3) NHANES paper (or something) ?) A simpler introduction: https://arxiv.org/abs/2504.08324?